CS4782 Final Report: Reimplementation of LoRA Test

Kevin Tang, Madur Malliah Ramkumar, Weitao Su
https://github.com/weitaosu/lora

1. Introduction

Finetuning is a common method of making a pre-trained model suitable for your specific use case. However, completing full finetuning of large models quickly drives up compute and memory requirements, often making it impractical or overly expensive. LoRA: Low-Rank Adaptation of Large Language Models, by Hu et al. (2021), attempts to solve these problems. The key idea is that for most tasks, instead of updating the full weight matrix W, LoRA freezes W and learns a low-rank update $\Delta W = BA$, where $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$, and $r \ll d$. If we set the rank r to a small value, we get a greatly reduced number of parameters to train, all while retaining equivalent performance.

2. Chosen Result

The results we focus on are the natural language generation (NLG) part of the paper. This focus is motivated by the fact that NLG requires the model to produce coherent, contextually appropriate text, making it a more demanding test of whether LoRA can effectively adapt large pretrained models without full fine-tuning. The model we chose was GPT-2 Medium (Radford et al., 2019), as it was small enough to finetune with our available compute, while still being large enough to test the capability of LoRAs on large models. The three datasets used for this model in the paper are E2E NLG Challenge (Novikova et al., 2017), DART (Nan et al., 2020), and WebNLG (Gardent et al., 2017), with the corresponding results in Appendix A. The metrics focused on were the BLEU and METEOR scores since they existed across all three datasets as a comparable measure for performance. Finally, we also aimed to measure the memory usage during training and number of trainable parameters as metrics important to explaining the benefits of LoRA.

3. Methodology

The general structure of the datasets followed a format of containing a 'structured meaning representation', as well as an accompanying 'target reference'. Training requires the concatenation of these two, but unlike the paper, we introduced a unique separator token "`<|SEP|>`" (Fig. 11). Empirically we observed that the model performed better with this configuration as opposed to the original paper, possibly due to this giving the model a clear understanding that whatever followed the separator token must be the target reference to be learnt. This token was also added to the GPT-2 Medium tokenizer and model vocabulary. Hyperparameter tuning wise, we use the same parameters reported by the paper as per Fig. 4, and per Li and Liang (2021).

We reimplement the LoRA modules by building the LoRA layers from scratch, and injecting them into the q and v layers of the GPT-2 Medium transformer model. We then perform a full fine-tuning of the baseline GPT-2 Medium model, and compare its performance against fine-tuning on the LoRA model. Key statistical (trainable parameter count, accuracy, loss) and hardware (peak GPU memory, throughput) performance were recorded, alongside all the evaluation metrics covered in the original paper for each dataset. A rank ablation study was also conducted to identify the best LoRA rank.

4. Results & Analysis

As can be observed from the tables in Appendix C, for every metric in every dataset, we were able to generally capture the same trend, that LoRA consistently either outperforms or achieves the same as the baseline fine-tuning model. Figures in Appendix D also show how the common BLEU and METEOR metrics behave across all datasets, once again supporting the above trend. Additionally, we were able to achieve a considerable decrease in peak GPU memory by 1.4 times. While this may be lesser than the paper's reduction by '3 times', it could likely be due to a difference in the accelerator used, as we did not have access to their V100 GPUs.

We also conducted a rank ablation study on several metrics, and present how validation loss behaves on different ranks in Figure 9. It is evident that beyond rank 4, we start seeing diminishing returns in performance, compared to the increase in trainable parameters (Fig. 8). Thus, we conclude that this is indeed a good middleground between model performance and size, supporting the original paper's findings.

One key difference between our reimplementation and the paper would be that while we were able to capture the same trends of LoRA outperforming or matching the baseline, all our models' numerical performances were slightly lower than the paper's reported numbers, within a 80 % margin. We attribute this to mainly undocumented configuration information in the LoRA paper. For instance, the original authors do not clearly document exactly how they preprocess the data for each dataset. Naturally this was also one of the biggest obstacles in reimplementing the paper faithfully. Similarly, the LoRA paper mentions to reference Li and Liang (2021)'s hyperparameter setup, but LoRA also provides some of its own conflicting parameters (Figs. 4 and 5).

5. Diffusion LoRA

As an extension to our LoRA reproduction on GPT-2 Medium + WebNLG, we tested whether the paper's central low-rank-suffices insight transfers from autoregressive language models to a 9-billion-parameter denoising diffusion transformer. Concretely, we trained a style LoRA on Flux.2 Klein 9B (Labs, 2025) + Qwen3-8B text encoder (frozen, qfloat8-quantized to fit in 24 GB VRAM on a single RTX 4090) using 77 manually-curated paintings by Werner Bronkhorst. The math is identical to the original paper: every linear layer W in the diffusion transformer's attention and MLP blocks gets a frozen weight plus a trainable low-rank update $\Delta W = (\alpha/\mathrm{r})\cdot B\cdot A$ with rank $r = 32$ and alpha $\alpha = r$. Trainable parameters: 165 M out of 9 B (≈ 1.8 %). The text encoder, VAE, and layer norms stay frozen entirely.

The key methodological difference from the LLM setting is the trigger word. Every caption was prefixed with "bronkhorst style, " so the Qwen3 embedding for that token carries the style signature, gating when the LoRA fires at inference. We trained for 5,000 steps (about 3 h) using ai-toolkit with AdamW8bit (lr 1e-4), flow-matching loss, EMA decay 0.99, gradient checkpointing, and bf16 mixed precision. Critical speed wins came from disabling aspect bucketing (one bucket means no CUDA recompile, 245× speedup) and pre-caching VAE latents and Qwen3 text embeddings to disk so both encoders could be unloaded from VRAM during training.

The largest practical challenges were not the hyperparameters. First, dataset quality was an issue. Manually filtering 271 scraped images down to 77 (a 28 % keep rate, dropping frame/wall/watermark/ wrong-artist contamination) outperformed all rank and step sweeps. Caption discipline was another problem. We had to strip style descriptors like "thick impasto" from captions so the trigger word alone carried the style signal. The final 165 MB LoRA produces strong, reliably-triggered style transfer that preserves subject fidelity across in-distribution and out-of-distribution prompts (lifeguards, knights, chefs, aerial scenes) as seen in Fig. 10, confirming that LoRA's low-rank insight generalizes cleanly from language transformers to diffusion transformers, provided the data and prompt pipeline around it are right.

6. Reflections

During this project, we feel that our main takeaway is being able to visually see how LoRA is able to reach full finetune performance. We also learned how to truly do a deep dive into a paper to fully understand how it works, digging through all the papers referenced to find the hyperparameters and dataset preprocessing setups. This was where we had the most issues, since the data preprocessing steps were rather unclear and varied between the LoRA paper, the Prefix Finetuning Paper that

LoRA references, and the actual dataset papers. At the very beginning, we had very bad results due to not properly formatting the inputs exactly like the paper did. Another issue was with the hyperparameter setups, where the LoRA paper's settings contradicts from Li and Liang (2021)'s paper in Figures 4 and 5.

One of the key limitations we found from the original approach was with the separator token. Since the LoRA paper used a simple " || " separator, which could have had previously learned meanings that made training less effective. Thus the major change we made was to exchange it for a newly created special token, which greatly helped performance especially in the E2E dataset.

If given more time and resources, we would've liked to focus on the GPT-3 model, since it was a major leap from GPT-2 and led directly into ChatGPT, with certain emergent properties. We would have liked to see if LoRA is able to adapt to these greatly increase parameter count properly, and retain its effectiveness at sizes of hundreds billions of parameters.

Some natural potential directions for this paper that we considered are DoRA, LoRA for different modalities and architectures, and LoRAs applied to individual experts in MoE models for adapting a single model to many applications at the same time. These all build upon the core idea of LoRA while expanding on its capabilities.

References

C. Gardent, A. Shimorina, S. Narayan, and L. Perez-Beltrachini. The webnlg challenge: Generating text from rdf data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pages 124–133, Santiago de Compostela, Spain, September 2017. Association for Computational Linguistics.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Black Forest Labs. Flux.2: Frontier visual intelligence. https://github.com/black-forest-labs/flux2, 2025.

X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.

L. Nan, D. Radev, R. Zhang, A. Rau, A. Sivaprasad, C. Hsieh, X. Tang, A. Vyas, N. Verma, P. Krishna, Y. Liu, N. Irwanto, J. Pan, F. Rahman, A. Zaidi, M. Mutuma, Y. Tarabar, A. Gupta, T. Yu, Y. C. Tan, X. V. Lin, C. Xiong, R. Socher, and N. F. Rajani. Dart: Open-domain structured data record to text generation. *arXiv preprint arXiv:2007.02871*, 2020.

J. Novikova, O. Dušek, and V. Rieser. The e2e dataset: New challenges for end-to-end generation. In *Proceedings of the SIGDIAL 2017 Conference*, pages 201–206, Saarbrücken, Germany, August 2017.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI*, 2019.

Appendix A. LoRA Paper Results

Model & Method	# Trainable Parameters	E2E NLG Challenge BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M ($\text{Adapter}^{\text{L}}$)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M ($\text{Adapter}^{\text{L}}$)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M ($\text{Adapter}^{\text{H}}$)	11.09M	$67.3_{\pm.6}$	$8.50_{\pm.07}$	$46.0_{\pm.2}$	$70.7_{\pm.2}$	$2.44_{\pm.01}$
GPT-2 M (FT^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.85}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{71.8}_{\pm.1}$	$\mathbf{2.53}_{\pm.02}$

Figure 1: LoRA paper's E2E results

Method	WebNLG BLEU↑ U	S	A	MET↑ U	S	A	TER↓ U	S	A
GPT-2 Medium									
Fine-Tune (354M)	27.7	**64.2**	46.5	.30	**.45**	.38	.76	**.33**	.53
$\text{Adapter}^{\text{L}}$ (0.37M)	45.1	54.5	50.2	.36	.39	.38	.46	.40	.43
$\text{Adapter}^{\text{L}}$ (11M)	**48.3**	60.4	54.9	**.38**	.43	**.41**	**.45**	.35	**.39**
FT^{Top2} (24M)	18.9	53.6	36.0	.23	.38	.31	.99	.49	.72
Prefix (0.35M)	45.6	62.9	55.1	**.38**	.44	**.41**	.49	.35	.40
LoRA (0.35M)	$46.7_{\pm.4}$	$62.1_{\pm.2}$	$\mathbf{55.3}_{\pm.2}$	**.38**	.44	**.41**	.46	**.33**	**.39**

Figure 2: LoRA paper's WebNLG results

Method	# Trainable Parameters	DART BLEU↑	MET↑	TER↓
GPT-2 Medium				
Fine-Tune	354M	46.2	**0.39**	**0.46**
$\text{Adapter}^{\text{L}}$	0.37M	42.4	0.36	0.48
$\text{Adapter}^{\text{L}}$	11M	45.2	0.38	**0.46**
FT^{Top2}	24M	41.0	0.34	0.56
PrefLayer	0.35M	46.4	0.38	**0.46**
LoRA	0.35M	$\mathbf{47.1}_{\pm.2}$	**0.39**	**0.46**

Figure 3: LoRA paper's DART results

Appendix B. Hyperparameters

Dataset	E2E	WebNLG	DART
		Training	
Optimizer		AdamW	
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size		8	
# Epoch		5	
Warmup Steps		500	
Learning Rate Schedule		Linear	
Label Smooth	0.1	0.1	0.0
Learning Rate		0.0002	
Adaptation		$r_q = r_v = 4$	
LoRA α		32	
		Inference	
Beam Size		10	
Length Penalty	0.9	0.8	0.8
no repeat ngram size		4	

Figure 4: LoRA Paper's LoRA Hyperparameters

	learning rate	# epoch	batch size	prefix length
Prefix:				
E2E	8e-05	5	10	5
WebNLG	5e-05	5	5	5
DART	5e-05	10	5	10
XSUM	5e-05	30	14	100
Adapter:				
E2E (3%)	5e-05	5	5	-
E2E (0.1%)	8e-05	10	5	
WebNLG (3%)	5e-05	5	5	-
WebNLG (0.1%)	5e-05	10	5	-
DART (3%)	5e-05	5	5	-
DART (0.1%)	8e-05	5	5	-
Fine-tune:				
E2E	5e-05	5	10	-
WebNLG	1e-05	10	6	-
DART	1e-05	10	6	-
FT-top2:				
E2E	5e-05	5	10	-
WebNLG	5e-05	10	9	-
DART	5e-05	5	5	-

Figure 5: Prefix-Tuning Paper's Baseline Full Finetune Hyperparameters

Appendix C. Reimplementation Results

Model	BLEU↑	NIST↑	METEOR↑	ROUGE-L↑	CIDEr↑
FT Paper	68.2	8.62	46.2	71.0	2.47
LoRA Paper	**70.4**	**8.85**	**46.8**	**71.8**	**2.53**
Our FT	65.5	**6.96**	45.0	64.7	1.73
Our LoRA	**65.8**	6.94	**45.6**	**65.2**	**1.81**

Table 1: Results from **E2E** Dataset. Higher is better for all metrics.

Model	BLEU↑			METEOR↑			TER↓		
	U	S	A	U	S	A	U	S	A
FT Paper	27.7	**64.2**	46.5	0.30	**0.45**	0.38	0.76	0.33	0.53
LoRA Paper	**46.7**	62.1	**55.3**	**0.38**	0.44	**0.41**	**0.46**	**0.33**	**0.39**
Our FT	14.0	**56.1**	41.3	0.17	**0.43**	0.33	0.85	**0.45**	0.60
Our LoRA	**40.3**	51.6	**47.5**	**0.36**	0.39	**0.38**	**0.57**	0.48	**0.51**

Table 2: Results from the **WebNLG** Dataset. Higher is better for BLEU and METEOR, while lower is better for TER.

Model	BLEU↑	METEOR↑	TER↓
FT Paper	46.2	0.39	0.46
LoRA Paper	**47.1**	**0.39**	**0.46**
Our FT	33.6	0.31	0.68
Our LoRA	**37.3**	**0.33**	**0.63**

Table 3: Results from the **DART** Dataset. Higher is better for BLEU and METEOR, while lower is better for TER.

Appendix D. Reimplementation Plots

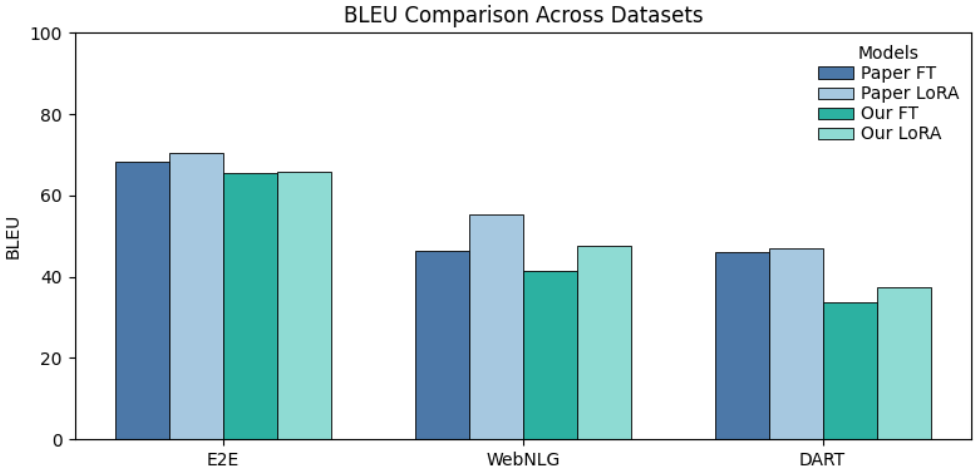

Figure 6: BLEU Score comparison between the paper's and our results, on both LoRA and Full Finetuning.

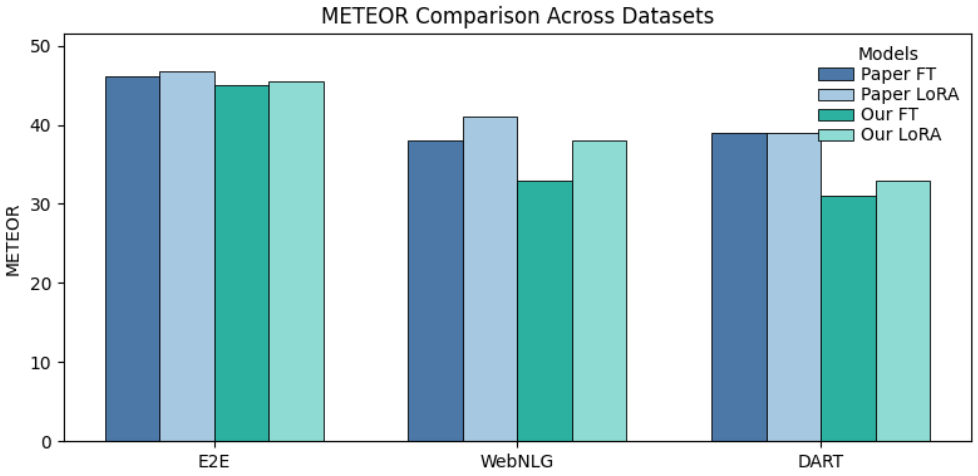

Figure 7: METEOR Score comparison between the paper's and our results, on both LoRA and Full Finetuning.

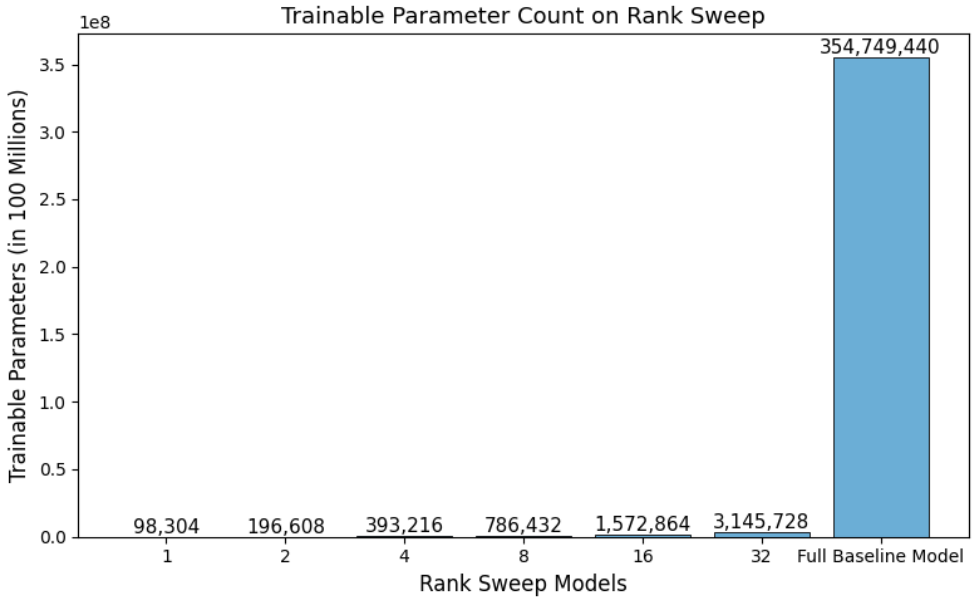

Figure 8: GPT-2 Medium Trainable Parameter Count with different LoRA ranks

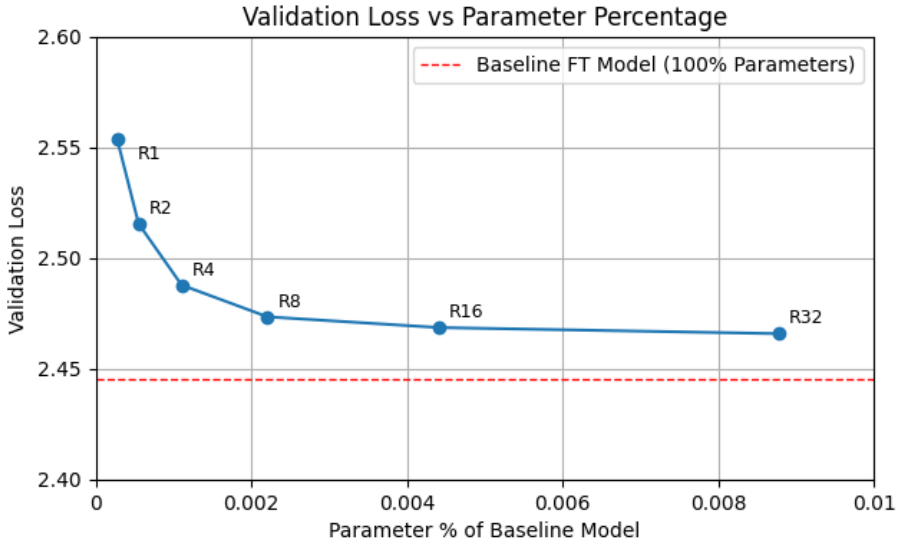

Figure 9: Validation Loss of LoRA model with different LoRA ranks

Appendix E. Diffusion LoRA

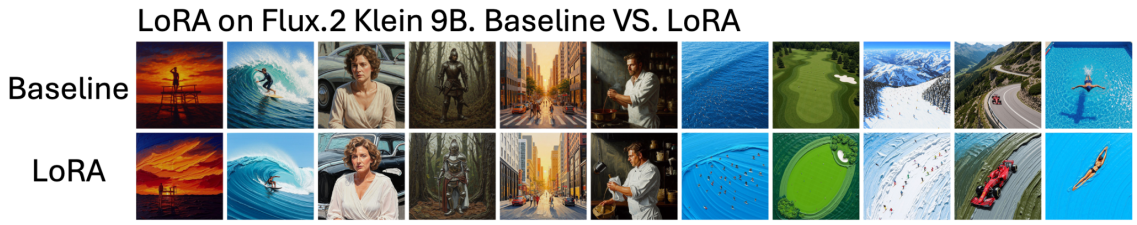

Figure 10: Comparison between base model and model with diffusion LoRA applied on the same prompts

Appendix F. Miscellaneous

Figure 11: Sample input representation of how structured meaning representation and target reference are concatenated